OpenAI Agents SDK: Approval Across Handoffs

Grounded DI | Public research edition | Local synthetic control research

Result in the tested configuration

The pinned SDK documents that sticky decisions for local ShellTool and ApplyPatchTool follow tool type and name across agents sharing approval state. In synthetic scripted handoffs, an always-approve decision on the first agent carried to a same-type, same-name tool on the second. Both in-memory recorders ran, and the destination policy and callback did not run. This is consistent with a documented configuration boundary, not a newly established defect.

Per-call shell approval and rejection, a different destination tool name, and fresh approval state all led the destination policy and rejecting callback to run; the second shell recorder did not run. A per-call ApplyPatchTool control likewise invoked the destination policy and rejecting callback. The editors only recorded the supplied synthetic operation and did not edit files.

An exact decision and a later call

In a separate shell fixture, the test script set an exact rejection for the second call before that call was emitted, while an always-approve decision existed. The second output recorded the synthetic rejection; a distinct third call reached the second recorder. This demonstrates the outcome of that specific test-injected state sequence. It is not a human decision made in response to a second interruption.

Why the configuration matters

The documentation explicitly recommends per-call decisions when executors or editors have different permissions and fresh approval state when starting a new run with different permissions. Its warning also says that a per-call decision does not clear an existing sticky decision. An application choosing sticky approval should understand that scope. The experiment does not show a production incident or unauthorized external action.

Cases and evidence boundaries

The audit-authored shell plan registered six cases: sticky handoff; per-call approve; per-call reject; different tool name; exact rejection followed by a later call; and fresh state. The last case exercised the destination directly rather than performing a handoff. A separately registered ApplyPatchTool plan added two handoff cases: sticky and per-call approval. Test-script event lists recorded decisions, policy and callback entries, and in-memory recorder invocations. No real shell or file mutation occurred.

The pinned source revision is e355d69152d4f4730dfb03b63dbce53e8d9f5d8c. The model was the SDK ScriptedModel, using fixed synthetic calls and identifiers. Results apply to the local library path and fixture. No hosted service, account, live model, independent external effect, or production deployment was tested. A tool or editor callback event is script-observed and not an independently monitored external effect.

Earlier populations and corrections

Eleven previously selected upstream parameterized cases passed; they are distinct from the eight new audit-authored cases. The prior v2 audit demonstrated shared approval state across separate runs, not a handoff. The DI² native baseline is separate again: the first attempt stopped at integrity because of an unlisted file; a clean copy recorded 99 implementation passes, six deliberate defect reproductions and 135 unit passes. The supplied lean harness verifier separately reported a manifest-format error.

In this round, the first shell run recorded five test-script errors because the recorder treated SDK call objects as dictionaries. A frozen recorder correction preceded the successful retry. The patch fixture first failed to import ApplyPatchResult; a frozen import correction preceded its retry. The initial outputs and amendments are retained privately. The earlier v2 wrong-context correction is preserved in its own package.

Isolation and limits

Telemetry was disabled before import and Python socket connect, connect_ex and create_connection were blocked; an OS-level network namespace was unavailable, and no independent network monitor was used. The re-created environment has recorded versions, but original distribution artifact hashes were not retained. The exact rejection was injected by the test script, not by an authenticated reviewer.

Reproduction and source

The private package retains the original v2 package, full locally archived pinned source, frozen plans, scripts, environment versions, raw outputs and checksums. The source is MIT licensed. The publicly available audit sources and documentation can be inspected at the links below.

Pinned OpenAI human-in-the-loop documentation

Selected prior upstream approval tests

What remains open

Further work could test genuinely independent reviewer identity, production application policy design, actual host-level execution and effects, and other configurations. Those questions require their own scope and authorization. The current local observations are not a security certification or a comparative ranking.